A directory to hold your manifests in the module is shown below:

```
<MODULE_PATH>/httpd/manifests/
```

A complete solution manifest is:

```
<MODULE_PATH>/httpd/manifests/init.pp
class httpd{
  include httpd::install
  include httpd::config
  include httpd::service
}
```

A manifest that just installs *httpd* is given below:

```
<MODULE_PATH>/httpd/manifests/install.pp
class httpd::install {  package {'httpd': ensure =>
'installed'}
}
```

A manifest that just configures *httpd* is given below:

```
<MODULE_PATH>/httpd/manifests/config.pp
class httpd::config{  file {'/etc/httpd/conf.d/httpd.conf':
    ensure => 'present',
    source => 'puppet:///modules/httpd/myhttpd.conf'
  }
}
```

A manifest that just handles *httpd* service is as follows:

```
<MODULE_PATH>/httpd/manifests/service.pp
class httpd::service{
  service{'httpd': ensure => 'running' }
}
```

Now, using it, we give the following command:

```
$ puppet apply --modulepath=<MODULE_PATH>  -e "include
httpd"
```

This will custom-configure, install and start the *httpd* service.

Apart from modules, we can also use classes and definitions in Puppet. Please refer to Figure 3 for the example for classes. Figure 4 gives the example of a definition with class.

Now, let's look at how to assign the resources and collections to client servers.

Puppet uses this file as a configuration file to determine which action should go to which client server. On client1 we can run the class *httpd* which contains

```
root@learning:~ # cat apa.install1
class apache {
        package { 'httpd':
                ensure => 'present',
                }
        service {'httpd':
           ensure => 'running',
           require => Package["httpd"],
           }
}
include apache
root@learning:~ # puppet apply apa.install
Notice: Compiled catalog for learning.puppet1kb.vm in environment production in 0
.seconds
Notice: Applied catalog in 35.22 seconds
root@learning:~ # rpm -qa | grep -i http
http-parser-devel-2.0-0.20111129gitcd01361.el7.x86_64
nodejs-http-signature-0.10.0-5.el7.noarch
httpd-2.4.6-40.el7.centos0.1.x86_64
```

Figure 3: Example of a Puppet class

```
root@learning:~ # cat aa.txt
class testdefine {
    define testdefine ($data) {
        file {"$title":
          ensure => file,
          content => $data,
        }
    }

    testdefine {'/var/testfile1':
      data => "The name of the file is puppetfile1 and it is created by puppet\n",
    }

    testdefine {'/var/testfile2':
      data => "The name of the file is puppetfile2 and it is created by puppet\n",
    }
    testdefine {'/var/testfile3':
      data => "The name of the file is puppetfile3 and it is created by puppet\n",
  }
}

include testdefine
root@learning:~ # puppet apply aa.txt
```

Figure 4: Example of a definition with class

the *httpd* service and package. On client2, it will run the Samba package and smb service.

The file *clients.pp* will look like what's shown below:

```
node 'client1.test.com' {
include httpd_class
}
node 'client2.test.com' {
include samba_class
}
node default {
    package { "perl":
       ensure => present }
}
```

In this article I have taken examples from many sites, especially Puppet Labs. There are many URLs from which we can get more examples of Puppet automation. Puppet is, indeed, a great tool to automate IT tasks. END

By: Ramasamy Panchavarnam

The author is a cloud architect at Sify. He has been in the IT industry since 1997, and is proficient in UNIX and cloud technologies. He is passionate about researching and testing open source tools, and can be reached at *sparcrams@yahoo.co.in*.